

The Modern Software Engineer: Design, Build, Learn

COMP-7431: Advanced Software Engineering | Week 1
IBM Design Thinking, the AI Periodic Table, and the semester ahead

We will build exciting software—but first we will learn to ask better questions.

Monday Morning: “Build Us an AI Chatbot”

THE FIRST REQUEST

“Students ask the same questions. Build a chatbot.”

WHAT THE NEW TEAM ACTUALLY SEES

- Five students describe five different problems
- Two campus offices publish conflicting instructions
- One student needs a voice-friendly experience
- Staff members repeatedly retype the same answers
- Nobody owns the accuracy of old web pages

The team has a technology idea—but it does not yet have a shared understanding of the human problem.

Week 1 Gives You the Map for the Semester

By the end of this lecture, you should be able to:

1. Describe how engineers move from a human need to a dependable system
2. Apply Design Thinking sequence: understand, find opportunities, envision, and prototype
3. Trace CampusConnect (your chatbot) from idea to a semester engineering story
4. Read the AI Periodic Table as a map of possible building blocks
5. Explain where RAG, agents, automation, and embedded systems may fit
6. Identify the Week 1 setup artifacts and the questions Week 2 must investigate

A Better Question Changes What We Can Build

SOLUTION-FIRST QUESTION

“How can we build a CampusConnect chatbot?”



HUMAN-PROBLEM QUESTION

“How can we help a student find trustworthy guidance and reach the right person without repeating the whole story?”

WHAT OPENS UP

- Better search or navigation
- A guided decision tree
- RAG with visible sources
- A voice option
- A staff handoff workflow
- A thoughtful combination of these ideas

The Developer's Job Has Become a Team Journey

Traditional habit	Modern software-engineering habit
Receive a finished specification	Join discovery and refine the need
Measure pages or code volume	Show small working slices and evidence
Choose the stack first	Compare simple and AI-assisted options
Create the repository later	Create the repository on day one
Keep one large private branch	Use small, reviewed branches
Test near the end	Define tests with the requirement
Treat deployment as the finish	Release, observe, and improve
Let automation stay invisible	Name triggers, failures, and owners
Treat a device as isolated	Design device, software, and human loop
Accept an AI answer	Inspect, test, revise, and log it

When Building Gets Faster, Choosing Matters More

A modern team can produce a first draft astonishingly quickly:

- Lovable or Bolt can turn a prompt into a working interface
- ChatGPT or Claude can draft stories, schemas, tests, and code
- ElevenLabs can add a voice experience
- An agent can call tools and update another system
- GitHub can preserve each issue, branch, review, and automated check
- Sensors can trigger software without a person clicking a button
- Automation can move data and launch the next action

Fast construction is exciting. It also makes it easier to build the wrong workflow, expose the wrong data, or automate a bad decision.

Design Thinking Gives Us Three Habits

1. Focus on human problems rather than starting with technical solutions
2. Explore options with people who bring different perspectives
3. Make mistakes early, while they are still inexpensive to correct

Mini example: sketch three ways a student could reach financial-aid help before anyone spends a week building the chatbot.

Design Thinking Frames AI as Help for People

WEAK STARTING STORY

“Replace the advisor with an AI.”



AUGMENTED-INTELLIGENCE STORY

“Help the student understand the next step, show the source, and bring in a person when the answer carries risk.”

WHAT SUCCESS COULD LOOK LIKE

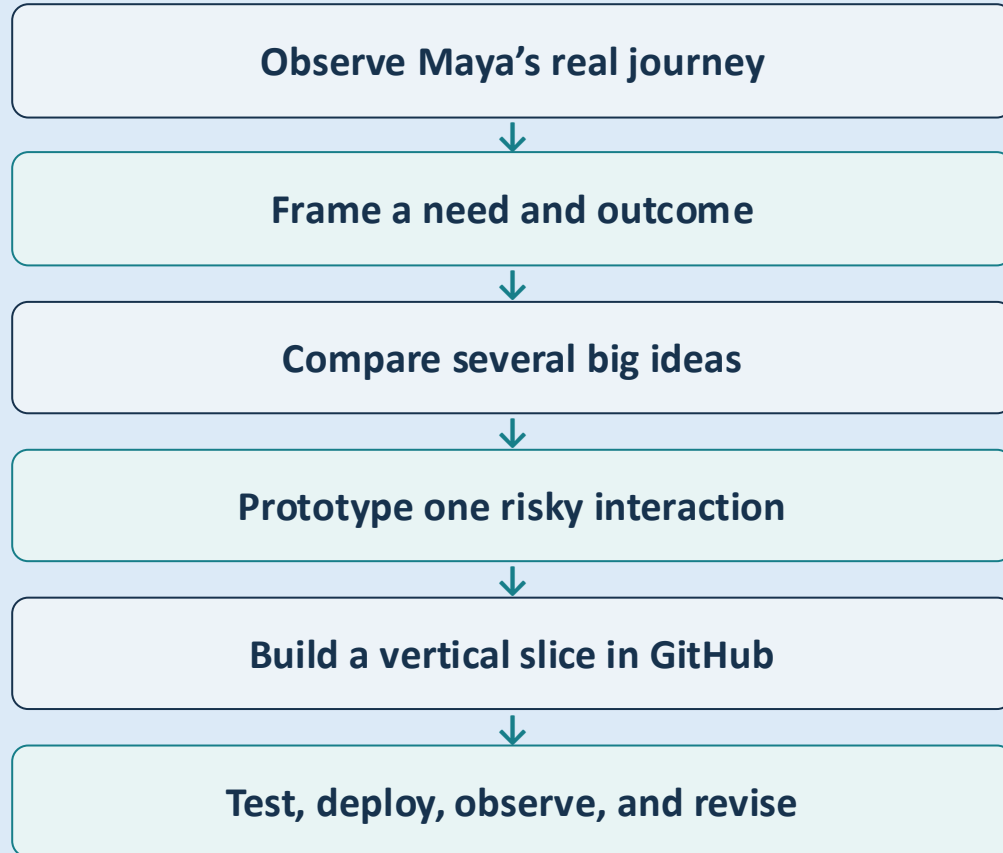
- The student stays in control of shared data
- The system explains where guidance came from
- Routine questions take less staff time
- Uncertain or sensitive questions reach a human
- Feedback improves the next version

The Semester Follows a Build-and-Learn Rhythm

1. Understand users
2. Map the current journey
3. Frame needs and outcomes
4. Set scope
5. Compare architectures
6. Design data and contracts
7. Prototype and validate
8. Implement and review
9. Test and run CI
10. Secure and red team
11. Deploy and observe
12. Improve and defend

The flow is not a one-way waterfall. Evidence from a prototype, test, or user can send the team back to an earlier decision.

One Student Story Can Travel the Whole Semester



THE STORY LEAVES EVIDENCE

- Interview and empathy notes
- As-is and to-be journeys
- Needs statement and requirements
- Architecture decision
- Prototype and user feedback
- Issues, commits, pull requests, and tests
- Engineering/AI Use Log
- Final demonstration and engineering defense

CampusConnect Begins with Maya, Not the Bot

MAYA'S THURSDAY AFTERNOON

“I found three pages, two phone numbers, and no clear answer about what I should do next.”

WHAT THE TEAM OBSERVES

- Maya does not use the university's department names
- She worries that asking the wrong office will delay aid
- One web page is current; another still ranks higher in search
- She wants a short answer, but she also wants proof
- If the answer is uncertain, she wants a warm handoff—not another link

A useful system must improve Maya's journey even when AI is uncertain or unavailable.

Understanding the User Starts with Humility

Here's a reminder: Your user is not you.

- Ellen wants digital grocery receipts because losing paper breaks her routine
- Jane needs weather information that fits a working parent's changing day
- Bill needs an insurance claim resolved without losing hours to coordination
- Maya needs trustworthy guidance without knowing the university's org chart
- A staff member needs answers that can be updated without editing code
- An accessibility reviewer needs the same journey to work by keyboard and voice

The team labels guesses as unknowns and brings them to real people in Week 2.

An Empathy Map Makes the Person More Complete

For one moment in Maya's journey, the team asks:

- SAY: "I do not want to miss a deadline."
- DO: Search, open several tabs, then call a general number
- THINK: "Does asking the wrong office make me look unprepared?"
- FEEL: Anxious, rushed, and uncertain
- VALUE: A clear next step and evidence that it is current
- UNKNOWN: Would Maya choose text, voice, search, or a human first?

Unknowns are not failures. They are the questions the team must investigate.

Map the Journey Before Repairing It

1. Notices a required form is missing
2. Searches in her own words
3. Opens three pages
4. Calls the general line
5. Repeats the story after transfer
6. Finally gets a next step

At every step:

DOING · THINKING · FEELING

The as-is map keeps the team from “solutioning” before it understands where the experience actually breaks.

Turn the Journey into a Needs Statement

TOO EARLY

“Maya needs a chatbot.”



IBM NEEDS-STATEMENT PATTERN

“Maya needs a way to identify a trustworthy next step so that she can act before the deadline without repeating her story.”

WHY THIS HELPS

- Names the person
- Describes the need as an action
- States the direct benefit
- Leaves several solutions open
- Gives Week 2 a claim to investigate

A Good System Serves More Than One Intent

Maya wants to...	The university needs to...
Find the right next step quickly	Reduce avoidable misrouting
Understand why the answer applies	Keep guidance traceable to an owner
Avoid repeating personal details	Collect only what the workflow requires
Use text, keyboard, or voice	Support accessible service channels
Reach a person when uncertainty is high	Route sensitive cases to trained staff
Know whether information is current	Review and retire outdated content

The student's intent and the system creator's intent can support each other—but they can also conflict.

Available Data Needs a Purpose

START WITH THE OBVIOUS

- Approved campus web pages
- Service directory
- Office hours and contact routes
- Search terms and anonymous failure patterns

CONSIDER WITH PERMISSION

- Calendar context
- Language or accessibility preferences
- Location for in-person services
- Status already shared with the service

DO NOT COLLECT “JUST IN CASE”

- Full transcripts without a purpose
- Credentials or secrets
- Private student records in prompts
- Personal data with no owner or retention rule

Data can create value only when the team can explain its source, permission, quality, owner, and role in the journey.

Jobs to Be Done Reveals Where AI Might Help

Choose one job: “Find the correct office and next step.”

1. Describe the question in the student’s words
2. Identify the topic and urgency
3. Find the most current approved guidance
4. Explain the next step in plain language
5. Show the source and owner
6. Offer a handoff when confidence is low
7. Preserve enough context so the student does not start over

Now mark the activities AI could improve. Do not mark every step merely because AI is available.

The AI Value Map Tests Whether AI Earns Its Place

Activity	Human work today	Possible AI value
Describe the need	Translate student wording	Recognize likely intent without requiring org-chart language
Find guidance	Search several sites	Retrieve passages from approved sources
Explain next step	Rewrite policy in plain language	Draft a concise answer with citations
Choose a route	Interpret topic and risk	Suggest a route; let rules or staff confirm high-risk cases
Prepare handoff	Ask the student to repeat details	Pass an approved summary with consent

Big Ideas Describe an Experience, Not a Button

FEATURE-FIRST IDEA

“Put a chatbot bubble on every campus page.”



BIG IDEA

“A student can begin anywhere, understand the next step, and reach the right person without starting over.”

OTHER BIG IDEAS TO SKETCH

- Guidance that always shows its owner and freshness
- A voice-friendly campus navigator
- A warm handoff that carries only approved context
- A staff cockpit that reveals unanswered questions
- A system that admits uncertainty before harm occurs

Choose the Next Move by Impact and Feasibility

Grid area	CampusConnect example
No Brainer	Show the source, owner, and last-reviewed date
Big Bet	RAG answer plus a context-preserving staff handoff
Utility	A searchable directory with clearer filters
Unwise—for now	An autonomous agent that changes student records

IBM's prioritization grid helps the team discuss ideas in relation to one another.

The To-Be Journey Weaves People, AI, and Data

MAYA'S ACTION

Explains the situation in her own words, reviews the proposed next step, opens the cited policy, and chooses self-service or a staff handoff.

SYSTEM AND AI

Interprets likely intent, retrieves approved passages, drafts a plain-language answer, checks risk, and prepares a bounded handoff.

DATA AND EVIDENCE

Uses reviewed campus content, records source and freshness, captures consent for handoff, and logs unanswered topics for improvement.

Read the new journey aloud. If it feels like a pile of features instead of one coherent experience, edit the story.

Prototype the Question, Not the Whole Product

Question to answer	Low-effort prototype
Can Maya understand the route?	Clickable Lovable or paper flow
Does a visible source increase trust?	Three sample answers with citations
Would voice help or slow the task?	Short ElevenLabs conversation
Can RAG find the right passage?	A few approved documents and test questions
Will the handoff preserve context?	A mocked ticket and staff playback
Can staff update content safely?	A simple owner-and-review workflow

Choose the lightest prototype that can answer the team's most important question.

A Hypothesis Turns Excitement into a Test

VAGUE AI CLAIM

“CampusConnect AI will answer every student question.”



TESTABLE HYPOTHESIS

“If CampusConnect retrieves from reviewed service pages and shows its sources, students will identify the correct next step more often than with today’s search results.”

WHAT THE TEAM MUST NAME

- User and creator value
- Required data
- The specific task or sub-flow
- A measure of improvement
- A safe fallback
- Evidence that would change the idea

The Modern Engineer Leaves a Reviewable Trail

Quick but fragile habit	Semester habit
Accept the requested feature	Investigate the need and current journey
Build the whole idea	Prototype the riskiest assumption first
Copy an AI-generated artifact	Inspect, revise, test, and explain it
Keep project files in personal folders	Use a shared repository and agreed project workspace
Describe work in chat	Open issues with owners and acceptance evidence
Demonstrate the happy path	Test boundaries, failures, usability, and recovery

The AI Periodic Table Maps Possible Building Blocks (click on the image)

Data Sources → Data Preparation → Representations → Model Learning → Model Families → Interaction Layer → System Integration → Agentic Design → Developer Frameworks → Automation Platforms → Applied AI Systems										
KNOWLEDGE FOUNDATION · COLUMNS 1–3			INTELLIGENCE ENGINE · COLUMNS 4–5		INTERACTION & INTEGRATION · COLUMNS 6–7		AGENTIC BUILD LAYER · COLUMNS 8–10			APPLIED AI SYSTEMS · COLUMN 11
KNOWLEDGE FOUNDATION Data → Preparation → Representation			INTELLIGENCE ENGINE Learning → Model Families		INTERACTION & INTEGRATION Prompts → APIs / Tools		AGENTIC BUILD LAYER Agents → Frameworks → Platforms			APPLIED AI SYSTEMS Real-world use cases
I DATA SOURCES	II DATA PREPARATION	III REPRESENTATIONS	IV MODEL LEARNING	V MODEL FAMILIES	VI INTERACTION LAYER	VII SYSTEM INTEGRATION	VIII AGENTIC DESIGN	IX DEVELOPER FRAMEWORKS	X AUTOMATION PLATFORMS	XI APPLIED AI SYSTEMS
Td Training Data The raw experience models learn from. 1.1	Tk Tokenization Text becomes model-readable pieces. 1.2	Em Embeddings Meaning becomes numbers. 1.3	Tr Training Models learn patterns from data. 1.4	Gp GPT Generative pretrained transformer family. 1.5	Pr Prompt Human instruction to the model. 1.6	Ap API Software talks to software. 1.7	Ag Agent AI with a goal and actions. 1.8	Lc LangChain Chains model, tools, retrieval. 1.9	N8 n8n Workflow automation platform. 1.10	Rb Robotics AI embodied in physical systems. 1.11
Sd Synthetic Data Artificially generated training examples. 2.1	Ch Chunking Large text becomes usable pieces. 2.2	Vs Vector Space A geometry of meaning. 2.3	Ft Fine-Tuning Specializing a model. 2.4	Cl Claude Anthropic model family. 2.5	Sp System Prompt Behavioral instructions. 2.6	Ep Endpoint A specific API destination. 2.7	G1 Goals What an agent is trying to do. 2.8	Li LlamaIndex Data-first LLM framework. 2.9	Zp Zapier No-code app automation. 2.10	Es Embedded Systems AI in devices and machines. 2.11
Dc Documents PDFs, webpages, policies, reports. 3.1	Cn Cleaning Removing noise and junk. 3.2	Vd Vector Database Stores meaning for retrieval. 3.3	R1 RLHF Human feedback improves behavior. 3.4	Gm Gemini Google multimodal model family. 3.5	Pe Prompt Engineering Designing better instructions. 3.6	Fn Function Calling Models call tools or code. 3.7	P1 Planning Breaking goals into steps. 3.8	Lg LangGraph Stateful agent workflows. 3.9	Mk Make Visual automation workflows. 3.10	Dt Digital Twins Virtual replicas of real systems. 3.11
Ia Images / Audio Multimodal raw material. 4.1	Ps Parsing Extracting structure from messy files. 4.2	Ss Similarity Search Find close meanings. 4.3	Ev Evaluation Measuring output quality. 4.4	Lm Llama Open-weight model family. 4.5	Ce Context Engineering Selecting the right information. 4.6	Mc MCP Model Context Protocol. 4.7	Tu Tool Use Agents use external capabilities. 4.8	Ca CrewAI Role-based agent teams. 4.9	Vf Voiceflow Conversational AI design. 4.10	De Digital Employees AI workers for business workflows. 4.11
Ed Enterprise Data CRM, emails, databases. 5.1	Nm Normalization Standardizing messy formats. 5.2	Rt Retrieval Find context before answering. 5.3	A1 Alignment Shaping behavior toward goals. 5.4	Ms Mistral Efficient open model family. 5.5	Gd Guardrails Boundaries for safer output. 5.6	Sd SDK Developer toolkit for APIs. 5.7	Ma Multi-Agent Specialized agents working together. 5.8	Au AutoGen Agent-to-agent conversations. 5.9	F1 Flowise Visual LLM workflow builder. 5.10	As AI Assistants Enterprise copilots and helpers. 5.11
Sn Sensor Data Physical-world signals. 6.1		Rg RAG Retrieval-Augmented Generation. 6.3		Gr Granite IBM enterprise model family. 6.5	Ts Temperature Creativity vs predictability. 6.6	Wh Webhooks Event-triggered communication. 6.7	Et Agent Ethics Autonomy requires accountability. 6.8		Cr Cursor AI coding environment. 6.10	Ao AI Operations Applied AI for business processes. 6.11

Read the Table as Five Connected Neighborhoods



Each neighborhood answers a different engineering question:

- Knowledge Foundation — source data and representations
- Intelligence Engine — model behavior and evaluation
- Interaction & Integration — prompts, APIs, and tools
- Agentic Build Layer — goals, plans, and automation
- Applied AI Systems — people, devices, and real workflows

The team travels through only the neighborhoods justified by the problem and the hypothesis.

The Table Should Trigger Engineering Questions

Periodic Table area	Question the CampusConnect team asks
Knowledge Foundation	Which approved documents become ground truth, and who keeps them current?
Intelligence Engine	Do we need a model, and what cases will expose weak behavior?
Interaction + Agentic Build	Which prompts, APIs, tools, goals, and approvals control the flow?
Applied AI Systems	How will this capability fit a real student and staff journey?

A table element earns its place only when it improves a justified outcome.

Automation Also Reaches the Physical World

SOFTWARE FLOW

1. Form triggers validation
2. Webhook creates an issue
3. n8n routes the work
4. API updates status
5. Human handles exceptions

EMBEDDED LOOP

1. Sensor reads air quality
2. Edge device filters noise
3. Software checks a threshold
4. Ventilation responds
5. Staff can see and override

Automation becomes an engineered system when triggers, data, decisions, actions, failures, and human authority are visible.

RAG Gives CampusConnect a Traceable Answer Path

THE STUDENT ASKS

“What do I do if a required financial-aid form is missing?” The wording does not match a page title or department name.

RAG RETRIEVES

The system searches chunks from approved documents, selects relevant passages, and gives the model only the evidence needed for this question.

THE ANSWER HELPS

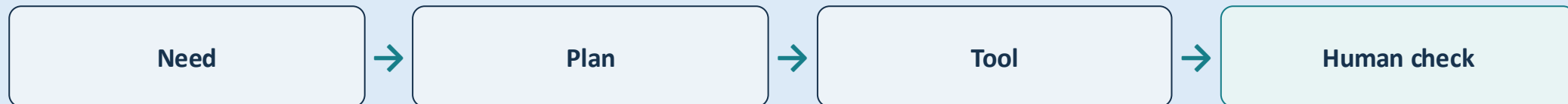
CampusConnect explains the next step, links the source, names the owner, displays freshness, and offers a human handoff when evidence conflicts.

RAG can reduce unsupported answers. It cannot repair outdated, conflicting, or ownerless content by itself.

An Agent Adds Goals, Tools, and Consequences

CampusConnect becomes agentic when it can pursue a goal through actions:

- Goal: help Maya reach the correct service
- Plan: identify topic, retrieve guidance, assess risk, offer the next step
- Tool use: query approved content or create a bounded handoff ticket
- State: remember which step has been completed
- Guardrail: never change a student record or expose private data
- Human control: require approval for consequential actions
- Evidence: log the tool call, result, failure, and owner



You Will Build with Both New and Proven Tools

The toolchain will change as the question changes:

- Lovable or Bolt — quickly build and revise a working interface
- ElevenLabs — explore a voice experience or agent conversation
- ChatGPT or Claude — draft, compare, explain, and critique engineering artifacts
- GitHub or GitLab — store code, issues, reviews, and the project history
- VS Code or Cursor — inspect and revise the actual implementation
- Postman and OpenAPI — make interface expectations executable
- pytest or JUnit — turn expected behavior into repeatable tests
- GitHub Actions or GitLab CI — repeat checks when the repository changes

AI builders accelerate the first slice. The repository remains the team's durable engineering record.

The Week 1 Lab Will Map the Semester

1. Discovery
2. User journey
3. Needs and outcomes
4. Requirements and scope
5. Architecture options
6. Data and contracts
7. Prototype and validation
8. Implementation and review
9. Tests and CI
10. Security and red team
11. Deployment and monitoring
12. Demo and defense

Students will leave the lab with one shared lifecycle map that becomes the navigation page for the semester project.

A Project Charter Helps the Team Start Well

PURPOSE AND BOUNDARY

- Human problem to investigate
- Intended users and stakeholders
- Initial outcome—not a locked feature list
- What is outside the first project boundary

WORKING AGREEMENTS

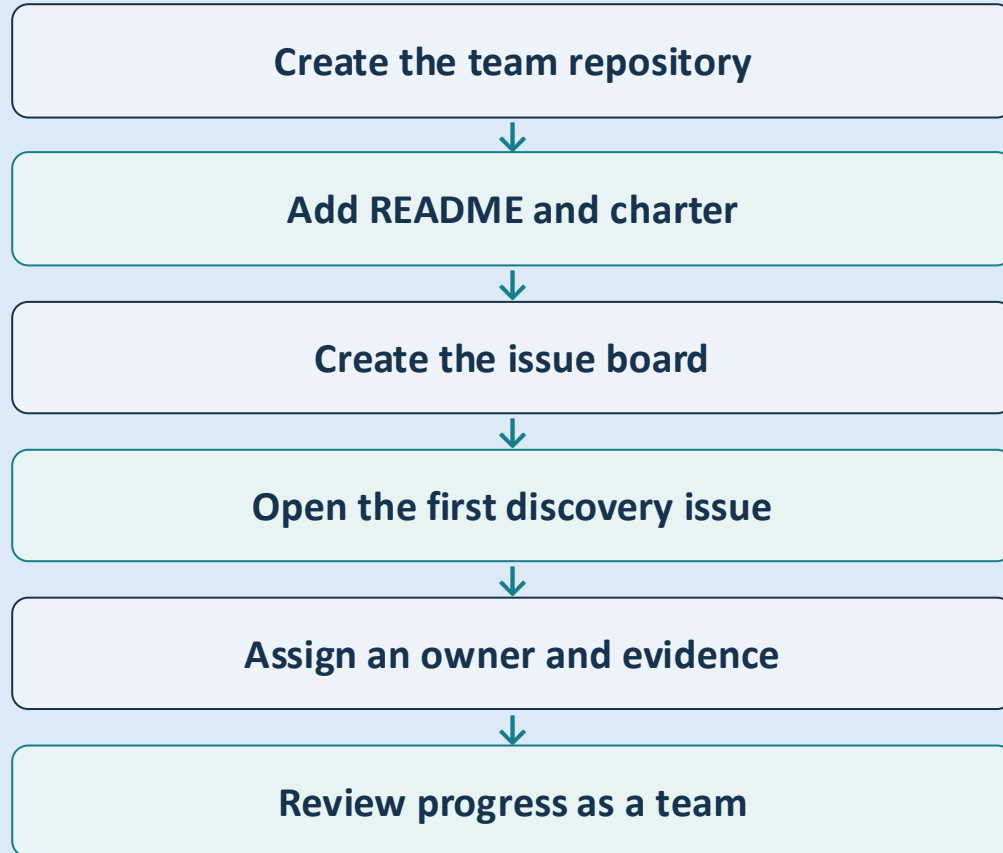
- Meeting and communication rhythm
- How work is assigned and reviewed
- How absences or blockers are raised
- Respectful disagreement and team care

DECISIONS AND QUALITY

- Who can approve scope changes
- What must receive peer review
- How AI-generated work is identified
- What evidence is needed before “done”

The charter is a short agreement the team can actually use—not a ceremonial document written once and forgotten.

Create the Engineering Trail on Day One



WHAT STUDENTS WILL DO

- Choose GitHub or GitLab
- Invite every teammate
- Protect the main branch
- Create four board views
- Open evidence-based issues
- Link later branches and reviews
- Exclude secrets and personal data

The Engineering/AI Use Log Makes Assistance Visible

When / artifact	AI assistance used	What the engineer verified or changed
Discovery summary	Claude grouped interview notes	Team checked every theme against the original notes
Needs statement	ChatGPT drafted three versions	Students removed solution language and kept the user benefit
Prototype	Lovable generated the first flow	Team revised accessibility, error states, and data fields
Code	Coding assistant drafted validation	Peer found an unsafe assumption and added tests
Test set	AI suggested edge cases	Team corrected expected results and added one missing failure path

Next Week: Investigate Before You Define

Week 2 focuses on Stakeholder Discovery and Problem Investigation:

- Identify primary, indirect, operational, and affected stakeholders
- Prepare interviews that invite stories instead of yes-or-no approval
- Observe the current workflow and exceptions
- Build empathy maps and as-is scenario maps from evidence
- Separate facts, assumptions, unknowns, and risks
- Look for unnecessary steps, confusion, delays, and pain points
- Write an initial needs statement that keeps multiple solutions open
- Decide which unanswered question the team must investigate next

CampusConnect does not need a feature backlog yet. It needs a more truthful picture of the people and the problem.

Your Semester Starts with a Better Question

“We do not begin by asking what AI can build. We begin by asking whose experience should become better.”

- Understand the person and current journey
- Keep several big ideas alive
- Prototype the riskiest belief early
- Use the AI Periodic Table to find relevant building blocks
- Turn the chosen idea into a shared repository story
- Build small working slices with modern tools
- Review AI-generated work as carefully as human work
- Test behavior, usability, and failure
- Automate with visible limits and human authority
- Deploy, observe, learn, and improve

Bring your curiosity, your skepticism, and your willingness to make an early mistake. We are going to build something worth defending.